

Challenge 1 description

BHU

Write a function:

```
string solution(vector<string> &words);
```

that, given an array `words` made of N strings, each of length K , returns a string of length K that differs from every string in the array **in at most one position**. The returned string should consist of lowercase English letters. If there are multiple such strings, the function can return any. If no such string exists, the function should return an empty string.

Examples:

1. Given `words = ["aaka", "aaka", "aaja", "aaxa", "aaba"]`, one of the correct results is "aaba". Note that "b" can be replaced by any other letter and the solution will remain correct.
2. Given `words = ["bay", "zaz", "bab"]`, the function should return "baz".
3. Given `words = ["aya", "aba", "abb", "zba"]`, the function should return "aba".
4. Given `words = ["zzz", "bcb", "zzc"]`, the function should return "" (empty string).

Write an **efficient** algorithm for the following assumptions:

- each string in the array `words` is of length K and consists only of lowercase English letters;
- N is an integer within the range $[2..100,000]$;
- K is an integer within the range $[1..100,000]$;
- $N * K$ is an integer within the range $[2..1,000,000]$.

Copyright 2009–2024 by Codility Limited. All Rights Reserved. Unauthorized copying, publication or disclosure prohibited.

Challenge 2 description

You are given two arrays, A and B, each made of N integers. They represent a grid with N columns and 2 rows, where A is the upper row and B is the lower row.

Your task is to go from the upper-left cell (represented by A[0]) to the bottom-right cell (represented by B[N-1]) moving only right and down, so that the maximum value over which you pass is as small as possible.

Write a function:

```
int solution(int A[], int B[], int N);
```

that, given two arrays of integers, A and B, of length N, returns the maximum value on the optimal path.

Examples:

1. Given A = [3, 4, 6], B = [6, 5, 4], the function should return 5. The optimal path is 3 → 4 → 5 → 4.

3	4	6
6	5	4

2. Given A = [1, 2, 1, 1, 1, 4], B = [1, 1, 1, 3, 1, 1], the function should return 2. The optimal path is shown in the image below.

1	2	1	1	1	4
1	1	1	3	1	1

3. Given A = [-5, -1, -3], B = [-5, 5, -2], the function should return -1. The optimal path is (-5) → (-1) → (-3) → (-2).

-5	-1	-3
-5	5	-2

Write a function:

```
int solution(int A[], int B[], int N);
```

that, given two arrays of integers, A and B, of length N, returns the maximum value on the optimal path.

Examples:

1. Given $A = [3, 4, 6]$, $B = [6, 5, 4]$, the function should return 5. The optimal path is $3 \rightarrow 4 \rightarrow 5 \rightarrow 4$.

3	4	6
6	5	4

2. Given $A = [1, 2, 1, 1, 1, 4]$, $B = [1, 1, 1, 3, 1, 1]$, the function should return 2. The optimal path is shown in the image below.

1	2	1	1	1	4
1	1	1	3	1	1

3. Given $A = [-5, -1, -3]$, $B = [-5, 5, -2]$, the function should return -1. The optimal path is $(-5) \rightarrow (-1) \rightarrow (-3) \rightarrow (-2)$.

-5	-1	-3
-5	5	-2

Write an **efficient** algorithm for the following assumptions:

- N is an integer within the range $[1..100,000]$;
- each element of arrays A and B is an integer within the range $[-1,000,000,000..1,000,000,000]$.

KGP

Challenge 1 description

Strings with long blocks of repeating characters take much less space if kept in a compressed representation. To obtain the compressed representation, we replace each segment of equal characters in the string with the number of characters in the segment followed by the character (for example, we replace segment "CCCC" with "4C"). To avoid increasing the size, we leave the one-letter segments unchanged (the compressed representation of "BC" is the same string – "BC").

For example, the compressed representation of the string "ABBBCCDDCCC" is "A3B2C2D3C", and the compressed representation of the string "AAAAAAAAAABXXAAAAAAAAA" is "11AB2X10A".

ABBBCCDDCCC → A3B2C2D3C

AAAAAAAAAABXXAAAAAAAAA → 11AB2X10A

Observe that, in the second example, if we removed the "BXX" segment from the middle of the word, we would obtain a much shorter compressed representation – "21A". In order to take advantage of this observation, we decided to modify our compression algorithm. Now, before compression, we remove exactly K consecutive letters from the input string. We would like to know the shortest compressed form that we can generate this way.

Write a function:

```
int solution(string &S, int K);
```

that, given a string S of length N and an integer K , returns the shortest possible length of the compressed representation of S after removing exactly K consecutive characters from S .

Examples:

of the compressed representation of S after removing exactly K consecutive characters from S.

Examples:

1. Given S = "ABBBCCDDCCC" and K = 3, the function should return 5, because after removing "DDC" from S, we are left with "ABBBCCCC", which compresses to a representation of length 5 - "A3B4C".

ABBBCCDDCCC → A3B4C

2. Given S = "AAAAAAAAAABXXAAAAAAAAA" and K = 3, the function should return 3, because after removing "BXX" from S, we are left with "AAAAAAAAA", which compresses to a representation of length 3 - "21A".

AAAAAAAAAABXXAAAAAAAAA → 21A

3. Given S = "ABCDDDEFG" and K = 2, the function should return 6, because after removing "EF" from S, we are left with "ABCDDDG", which compresses to a representation of length 6 - "ABC3DG".

ABCDDDEFG → ABC3DG

Write an **efficient** algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- K is an integer within the range [0..100,000];
- $K \leq N$;
- string S is made only of uppercase letters (A-Z).

Challenge 2 description

There is a string of length N made only of letters "a". Whenever there are two identical adjacent letters (e.g. "aa"), they can be transformed into a single letter that is the next letter of the alphabet. For example, "aa" can be transformed into "b" and "ee" into "f". However, "zz" cannot be further transformed.

What is the alphabetically largest string that can be obtained from the initial string?

Write a function:

```
string solution(int N);
```

that, given an integer N , returns the alphabetically largest string that can be obtained after such transformations.

Examples:

1. Given $N = 11$, the function should return "dba". The initial string "aaaaaaaaaaa" can be transformed in the following manner: "aaaaaaaaaaa" $\rightarrow$ "bbbbba" $\rightarrow$ "ccba" $\rightarrow$ "dba".
2. Given $N = 1$, the function should return "a". The initial string "a" cannot be transformed in any way.
3. Given $N = 67108876$, the function should return "zzdc".

Write an **efficient** algorithm for the following assumptions:

- N is an integer within the range $[1..1,000,000,000]$.

Copyright 2009–2024 by Codility Limited. All Rights Reserved. Unauthorized copying, publication or disclosure prohibited.

MADRAS

Challenge 1 description

There is an array A of N integers. What is the largest sum of two numbers which do not have any common digits?

Write a function:

```
int solution(vector<int> &A);
```

that, given an array A of length N, returns largest possible sum of two numbers from A which do not have any digits in common. If it is impossible to choose two such numbers, the function should return -1.

Examples:

1. Given A = [53, 1, 36, 103, 53, 5], the function should return 108, because we could choose numbers 103 and 5. Other pairs of numbers which do not share any digits, for example 5 and 36, total to a smaller sum.
2. Given A = [9, 19, 99, 29], the function should return -1, as every pair of numbers shares the digit 9.
3. Given A = [6, 17, 71, 66, 17, 6], the function should return 137. The best pair is 71 and 66.
4. Given A = [15, 0, 105], the function should return 15.

Write an **efficient** algorithm for the following assumptions:

- N is an integer within the range [2..200,000];
- each element of array A is an integer within the range [0..1,000,000,000].



Challenge 2 description

In this task you are given two strings of digits that represent (possibly very large) integers. Your goal is to make those numbers as close to one another as possible. In other words, you want to minimize the absolute value of their difference. You can swap some of the corresponding digits (e.g. the first digit of the first number with the first digit of the second number, the second digit of the first number with the second digit of the second number, etc.). Swapping the digits is an extremely tiring task, so you want to make as few swaps as possible.

Write a function:

```
int solution(char *S, char *T);
```

that, given two strings *S* and *T*, both of length *N*, returns the minimum number of swaps needed to minimize the difference between the two numbers represented by the input strings.

For example, given *S*="29162" and *T*="10524" your function should return 2. We can swap the second and the fourth digits and obtain "20122" and "19564". The difference between the numbers is 558 and the number of swaps is 2. One can easily check that the difference is the smallest possible. Note that we could obtain the same difference by swapping the first, third and fifth digits, but this solution requires three swaps.

Write an **efficient** algorithm for the following assumptions:

- lengths of *S* and *T* are equal and within the range [1..100,000];
- *S* and *T* consist only of digits and no other characters;
- neither *S* nor *T* contain leading zeroes.

ROORKEE

Task 1

Task 2

Challenge 1 description

There is an array A of N integers and two types of operations that can be performed on the elements of the array:

- increment a single element of A , which costs $C1$;
- increment two elements of A , which costs $C2$. The chosen elements need to be in different positions.

What is the minimum total cost of operations that will make all elements of A equal? As the result may be large, return its last nine digits without leading zeros (in other words, return the result modulo 10^9).

Write a function:

```
int solution(int A[], int N, int C1, int C2);
```

that, given an array A of N integers and two integers $C1$ and $C2$, returns the minimum cost of equalizing the array (modulo 10^9).

Examples:

1. Given $A = [1, 4]$, $C1 = 15$ and $C2 = 3$, the function should return 45. We may increment the first element three times.
2. Given $A = [2, 11, 11, 11, 12]$, $C1 = 10$ and $C2 = 4$, the function should return



28°C
Haze



Search





Task 1

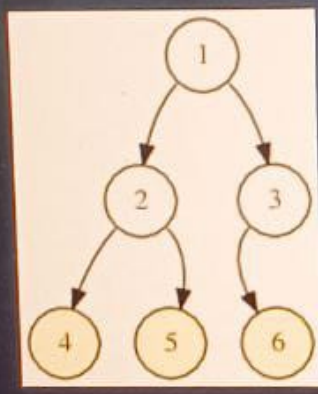
Task 2

Challenge 2 description

The problem focuses on a data structure called binary tree.

A node of a *binary tree* has an ID (an integer value) and pointers to two other nodes, called the *left subtree* and the *right subtree*. If the *left subtree* or *right subtree* does not exist, its corresponding pointer value is NULL.

For example, the figure below shows a binary tree consisting of six nodes. Here, node 3 does not have a right subtree, and nodes 4, 5 and 6 are *leaves* they do not have any subtree.



Assume that the following declarations are given:

```
struct tree { int x; struct tree * l; struct tree *
```

28°C
Haze



Search



Esc

F1

F2

F3

F4



Task 1

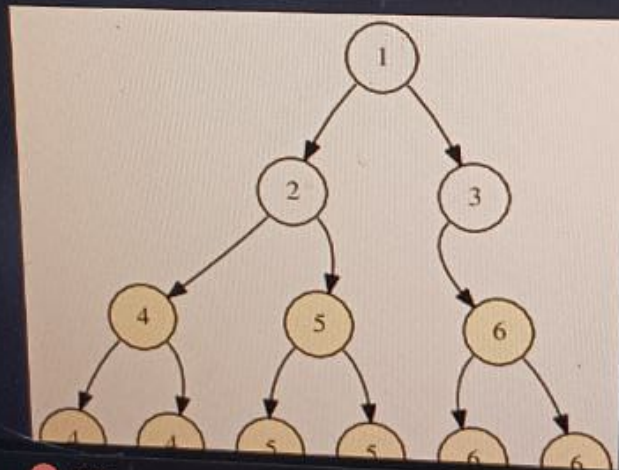
Task 2

Assume that the following declarations are given:

```
from dataclasses import dataclass, field @dataclass
class Tree: x: int = 0 l: "Tree" = None r: "Tree" =
None
```

Attribute *x* holds the node's ID, whereas attributes *l* and *r* hold the *left* and *right subtree*, respectively.

Your task is to *expand* the tree by replacing every *leaf* with a new node whose subtrees are both leaves. The IDs of each new node and its two leaves should correspond to the ID of the leaf they replaced. Here is the expanded tree from the previous example:



22°C
Haze



Search





Task 1

Task 2

Write a function:

```
def solution(T)
```

that, given a binary tree T consisting of N nodes, returns its expanded version.

Examples:

1. Given a tree T from the example above, the function should return the tree shown above in the second figure.

2. Given a tree T :



the function should return the tree shown in the following figure:



22°C
Haze



Search

